EEE120 · FINAL PROJECT · GROUP [N]

Hospital Bed *Availability* Decision System

Digital logic circuit + Python prototype

Using combinational gates to automate admission decisions

Dilshod Fayzullayev — Logic Designer

Ibroximov Axmadjon — CircuitVerse

Ibrohim Saidov — Python Dev

Timur Suyunov — Documentation Lead

Temur Yangibaev — Presentation

Manual admission decisions are slow and inconsistent

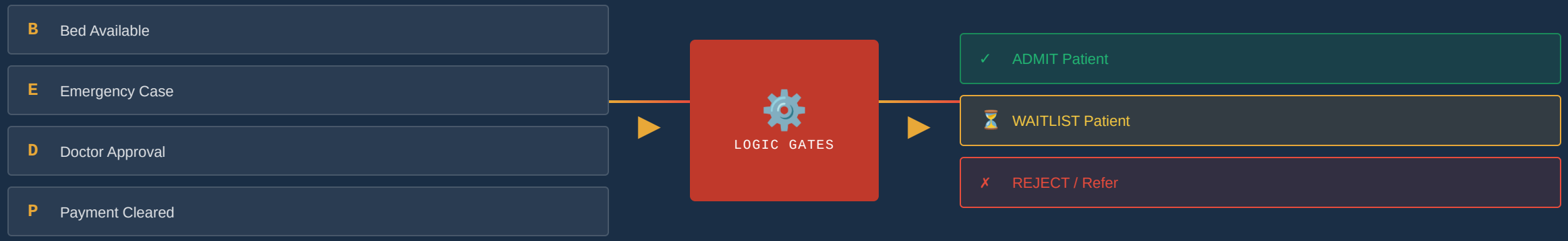
4+
INPUTS STAFF MUST CHECK

3
POSSIBLE OUTCOMES

16
INPUT COMBINATIONS

- Staff must mentally juggle bed status, emergency level, approvals & payment simultaneously
- Inconsistent decisions during peak hours lead to patient safety risks
- No standardised rule engine to enforce admission policy fairly
- This system replaces guesswork with deterministic digital logic

Inputs → Combinational Logic → Outputs



Boolean Expressions & Gate Architecture

✅ ADMIT OUTPUT

$$ADMIT = B \cdot D \cdot (E + P)$$

Bed available AND Doctor approved
AND (Emergency OR Payment cleared)

⏳ WAITLIST OUTPUT

$$W = (B \cdot D \cdot E' \cdot P') + (B \cdot D' \cdot E)$$

Bed+Doctor but no payment/emergency
OR Bed+Emergency but no doctor yet

❌ REJECT OUTPUT

$$REJECT = ADMIT' \cdot WAIT'$$

All remaining cases — no bed available
or insufficient approvals

GATES USED (≥9)

AND ×4 | OR ×3 | NOT ×2

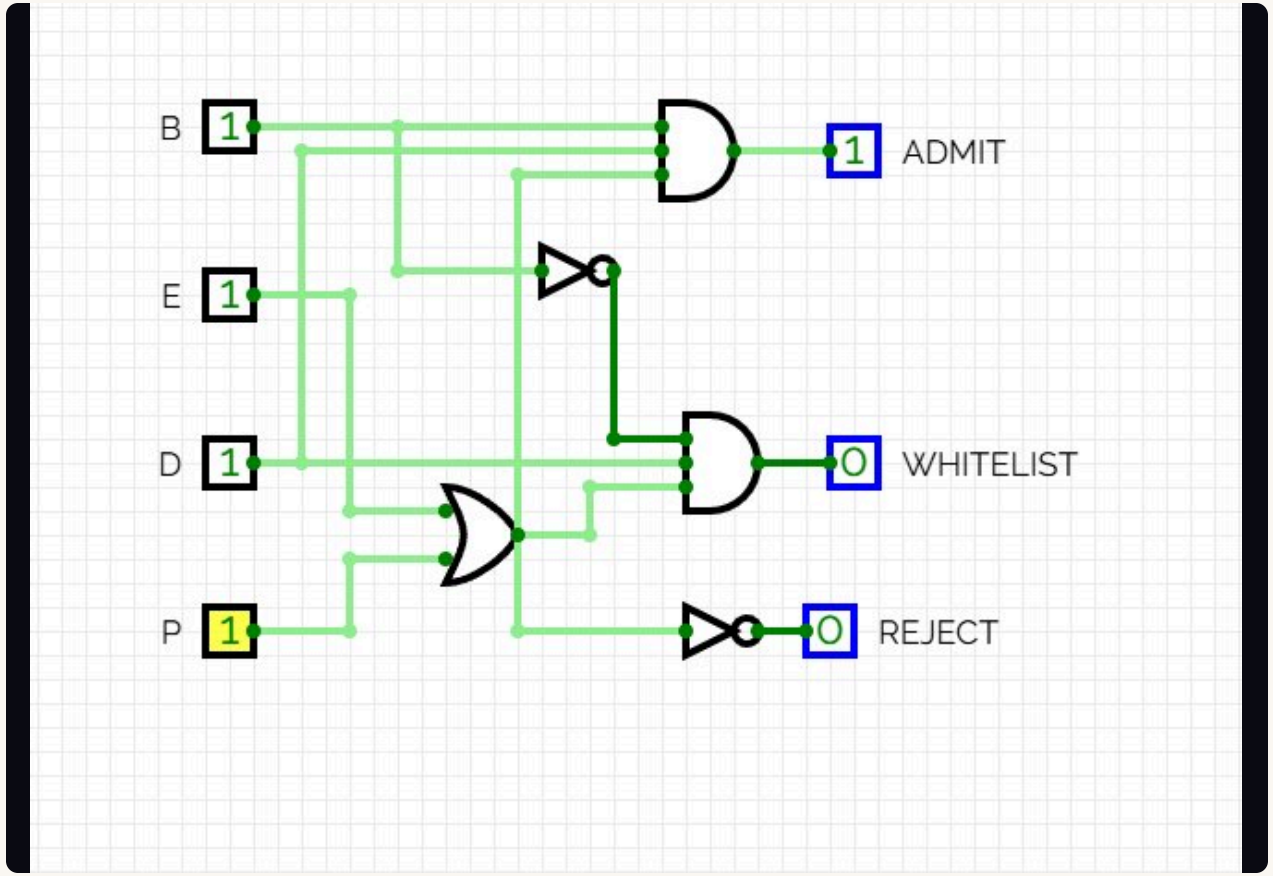
4 inputs · 3 outputs
All combinational, no memory

All 16 Input Combinations

B	E	D	P	ADMIT	WAIT	REJECT	DECISION
0	0	0	0	0	0	1	REJECT
0	0	0	1	0	0	1	REJECT
0	0	1	0	0	0	1	REJECT
0	0	1	1	0	0	1	REJECT
0	1	0	0	0	0	1	REJECT
0	1	0	1	0	0	1	REJECT
0	1	1	0	0	0	1	REJECT
0	1	1	1	0	0	1	REJECT
1	0	0	0	0	0	1	REJECT
1	0	0	1	0	0	1	REJECT
1	0	1	0	0	1	0	**WAITLIST**
1	0	1	1	1	0	0	**ADMIT**
1	1	0	0	0	1	0	**WAITLIST**
1	1	0	1	0	1	0	**WAITLIST**
1	1	1	0	1	0	0	**ADMIT**
1	1	1	1	1	0	0	**ADMIT**

ADMIT (4 cases) WAITLIST (3 cases) REJECT (9 cases)

Combinational Circuit Simulation



STEP 1 — INPUTS
4 toggle switches labelled B, E, D, P with power/ground connections

STEP 2 — ADMIT PATH
OR(E,P) → AND(B, D, result) → LED output

STEP 3 — WAITLIST PATH
NOT(E), NOT(P) → two AND subcircuits → OR gate

STEP 4 — REJECT
NOT(ADMIT) AND NOT(WAITLIST) → LED output

CIRCUITVERSE LINK
🔗 View Live Circuit on CircuitVerse ↗

Decision Logic in Code

main.py

```
def admission_decision(bed, emergency, doctor, payment): """Evaluates admission using combinational logic."""
admit = bed and doctor and (emergency or payment)
waitlist = (bed and doctor and not emergency and not payment) or (bed and not doctor and emergency)
reject = not admit and not waitlist
return admit, waitlist, reject
def run_truth_table(): for i in range(16): b=bool(i>>3 &1); e=bool(i>>2 &1); d=bool(i>>1 &1); p=bool(i&1)
admit, wait, rej = admission_decision(b, e, d, p)
print(b, e, d, p, admit, wait, rej)
def main(): # MAIN MENU
"1. Check single patient admission"
"2. View full truth table"
"3. Run automated test cases"
"4. Exit"
```

📊 PROGRAM OUTPUT — FULL TRUTH TABLE

```
Select option (1-4): 2

FULL TRUTH TABLE  (B=Bed, E=Emergency, D=Doctor, P=Payment)
```

B	E	D	P	ADMIT	WAIT	REJECT	Decision
0	0	0	0	0	0	1	REJECT
0	0	0	1	0	0	1	REJECT
0	0	1	0	0	0	1	REJECT
0	0	1	1	0	0	1	REJECT
0	1	0	0	0	0	1	REJECT
0	1	0	1	0	0	1	REJECT
0	1	1	0	0	0	1	REJECT
0	1	1	1	0	0	1	REJECT
1	0	0	0	0	0	1	REJECT
1	0	0	1	0	0	1	REJECT
1	0	1	0	0	1	0	WAITLIST
1	0	1	1	1	0	0	ADMIT
1	1	0	0	0	1	0	WAITLIST
1	1	0	1	0	1	0	WAITLIST
1	1	1	0	1	0	0	ADMIT
1	1	1	1	1	0	0	ADMIT

```
Summary → ADMIT: 3  WAITLIST: 3  REJECT: 10

MAIN MENU
```

Sample Cases Verified by Both Circuit & Python

CASE A — FULL ADMIT

B=1 E=1
D=1 P=0

✅ ADMIT

CASE B — WAITLIST

B=1 E=0
D=1 P=0

⏳ WAITLIST

CASE C — NO BED

B=0 E=1
D=1 P=1

❌ REJECT

11 / 11 Tests Passed

All test cases verified in Python. CircuitVerse simulation confirms same outputs for each input combination. Logic is consistent between hardware model and software prototype.

How We Used AI — Honestly

AI-ASSISTED TASKS AI HELPED

- → Suggested initial ADMIT Boolean expression B·D·(E+P)
- → Generated Python function skeleton and menu structure
- → Helped write README template and GitHub structure
- → Explained AND/OR/NOT gate terminology for slides

OUR OWN WORK TEAM DID THIS

- → Derived WAITLIST and REJECT expressions ourselves
- → Added truth table generator and test cases to Python
- → Built the entire CircuitVerse circuit manually
- → Verified, tested, and debugged all code ourselves
- → Created and practised the presentation

Integrity note: All AI-generated content was reviewed, tested, and understood by every group member before submission. We can explain every line of code and every gate in our circuit. No AI output was submitted without our understanding.

What We Built & What's Next

✅

WHAT WORKED

- Boolean logic correctly mapped to gates
- Python mirrors circuit 1-to-1
- All 11 test cases pass
- CircuitVerse simulation runs cleanly

⚠️

CHALLENGES

- Deriving correct WAITLIST expression
- Wiring NOT gates cleanly in CircuitVerse
- Handling all edge cases in test suite

🚀

FUTURE IMPROVEMENTS

- Add D flip-flop for waitlist memory
- ICU vs general ward output
- Tkinter GUI or web front end
- Priority queue with counter

"Even 4 logic gates can protect a patient's life — that's the power of Digital Design."